SYSTEM ARCHITECT AUTOMOTIVE

TÀI LIỆU CHUYÊN SÂU AUTOSAR CANTP & TÍCH HỢP COM STACK

Kịch bản tích hợp hệ thống và phân tích thuật toán chi tiết

CanTp

Biên soạn: **Phạm Văn Vũ**

HALA Academy

TÀI LIỆU CHUYÊN SÂU AUTOSAR CANTP & TÍCH HỢP COM STACK

Tài liệu này bám sát chi tiết mã nguồn trong repository, phân tích toàn bộ luồng xử lý từ các tầng ứng dụng (COM/Dcm) xuống phần cứng và ngược lại. Đồng thời, tài liệu cung cấp phân tích thuật toán chuyên sâu từng bit trong module CanTp (ISO 15765-2).

Bản chất cốt lõi của CanTp: CanTp chịu trách nhiệm cắt nhỏ các thông điệp chẩn đoán UDS hoặc PDU dữ liệu lớn (lớn hơn 8 bytes của CAN Classic) thành các mảnh tin nhỏ hơn tại phía gửi, và lắp ráp chúng trở lại nguyên vẹn tại phía nhận qua cơ chế điều khiển luồng (Flow Control CTS) một cách an toàn và tối ưu RAM.

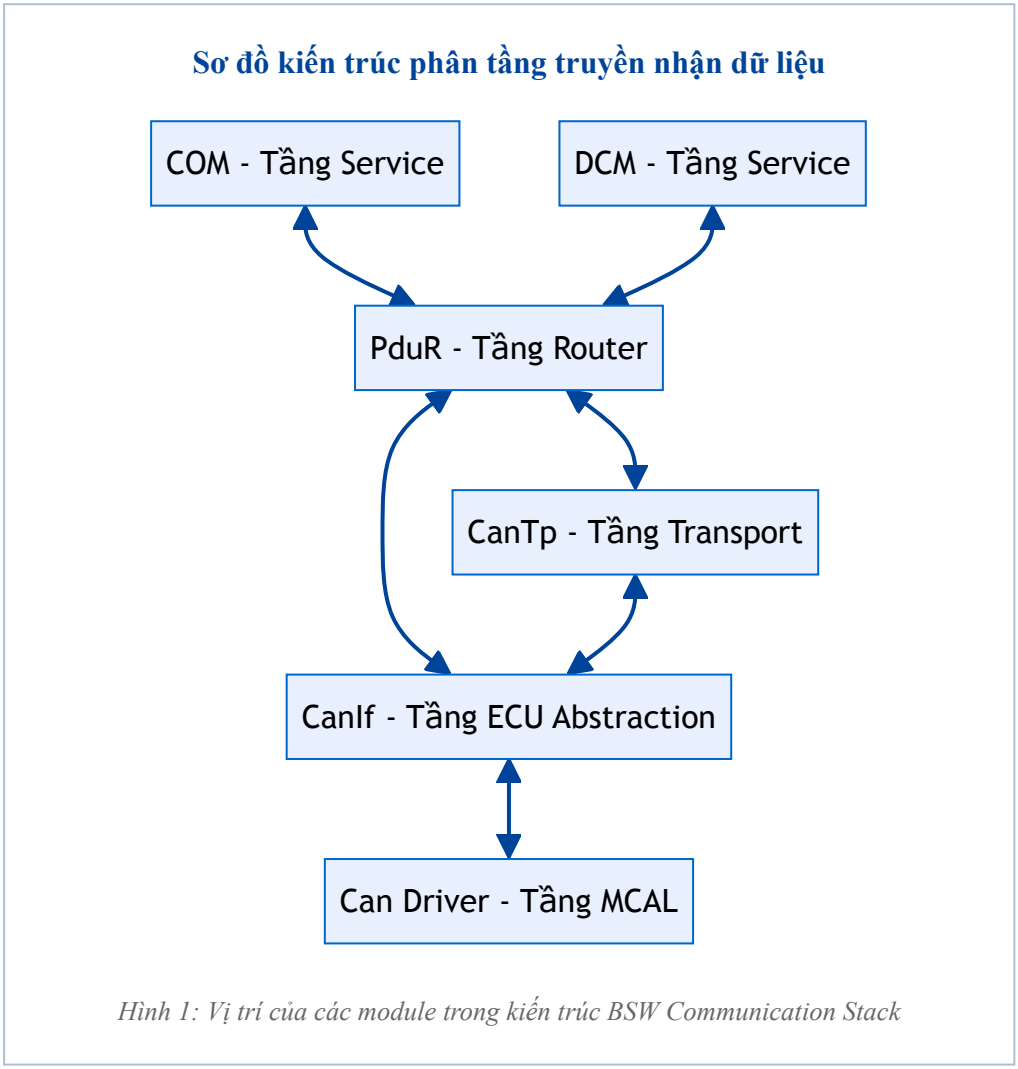
Mục lục

1. Bản đồ tương tác giữa các lớp (Call Stack Map)
2. Kịch bản 1: Truyền tín hiệu CAN cổ điển nhỏ từ COM
3. Kịch bản 2: Truyền chẩn đoán dữ liệu lớn phân mảnh qua CanTp
4. Kịch bản 3: Nhận chẩn đoán dữ liệu lớn phân mảnh qua CanTp
5. Thuật toán trích xuất & xử lý bitwise byte điều khiển PCI
6. Thuật toán phân mảnh dữ liệu phía phát (TX Segmentation)
7. Thuật toán lắp ráp dữ liệu phía nhận (RX Reassembly)
8. So sánh logic code hiện tại và tiêu chuẩn ISO 15765-2

---

1. Bản đồ tương tác giữa các lớp (Call Stack Map)

Trong kiến trúc AUTOSAR Classic COM Stack, các module trao đổi dữ liệu qua các hàm API đồng bộ và bất đồng bộ theo luồng phân tầng rõ ràng từ trên xuống dưới (gửi) và từ dưới lên trên (nhận):



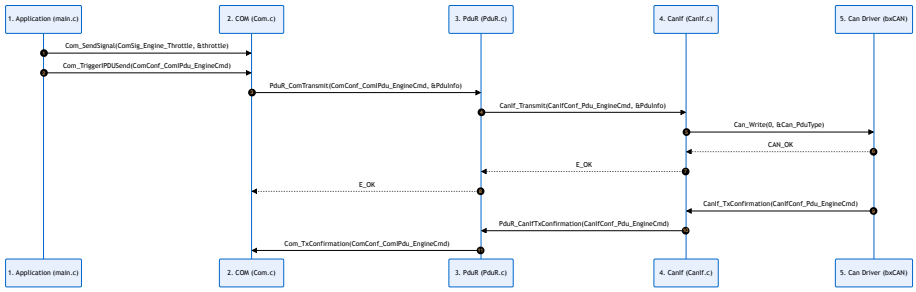
Hình 1: Vị trí của các module trong kiến trúc BSW Communication Stack

---

2. Kịch bản 1: Truyền tín hiệu CAN cổ điển nhỏ từ COM

Kịch bản này mô tả luồng gửi tín hiệu động cơ `EngineCmd` (CAN ID `0x180`, DLC = 5 bytes) từ ứng dụng đi xuống phần cứng CAN driver.

Sơ đồ tuần tự truyền dữ liệu nhỏ từ COM



Hình 2: Sơ đồ tuần tự truyền tín hiệu CAN từ tầng COM

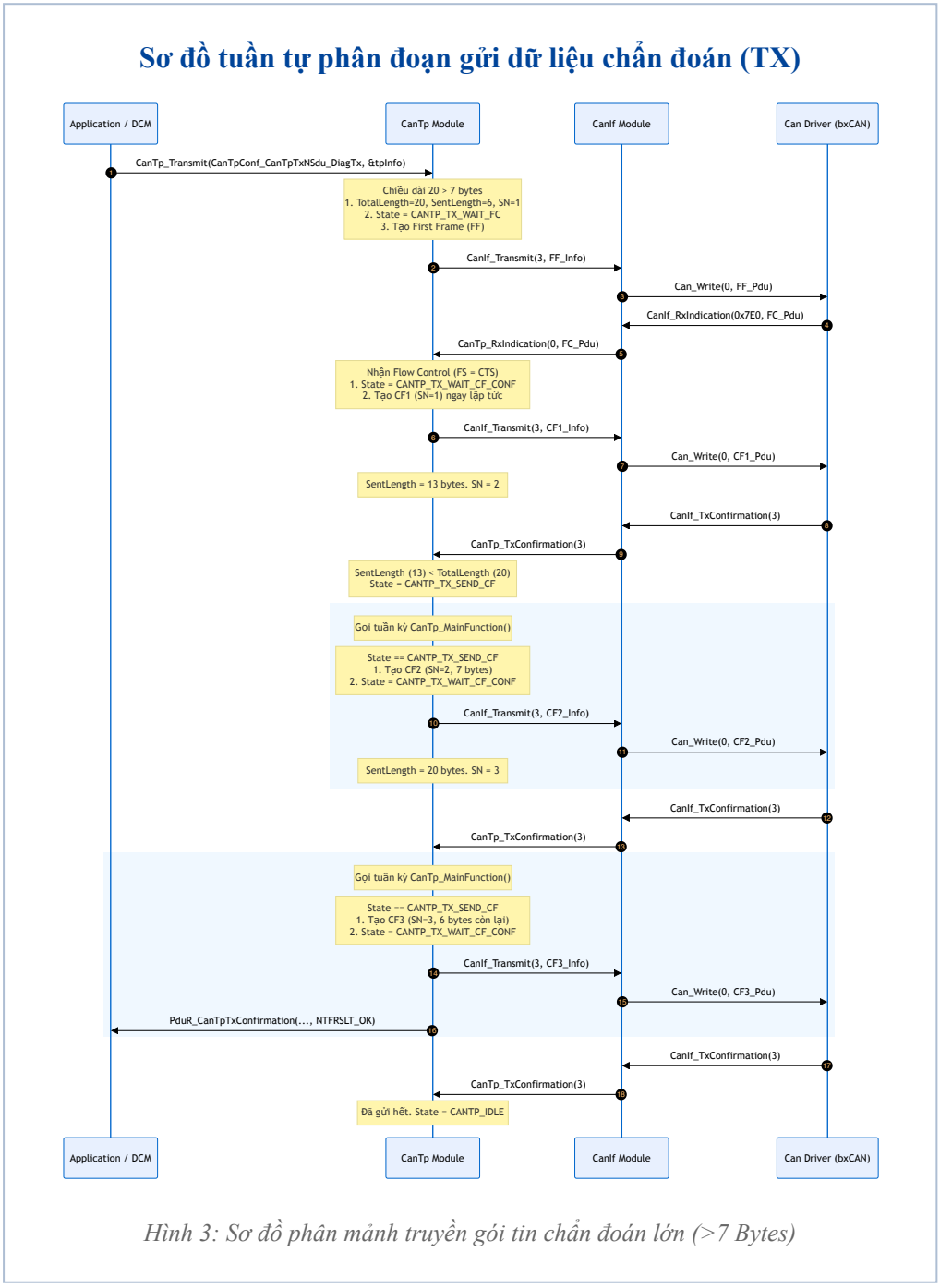
Mô tả chi tiết luồng xử lý:

1. Ứng dụng gọi `Com_SendSignal()` để lưu giá trị bướm ga vào shadow buffer của COM, sau đó gọi `Com_TriggerIPDUSend()` để kích hoạt việc gửi PDU chứa tín hiệu đó đi.
2. Tầng `Com` đóng gói các tín hiệu thành mảng byte thô rồi gọi `PduR_ComTransmit()`.
3. `PduR` tra bảng định tuyến tĩnh `PduR_ComTxRoutes` và chuyển tiếp yêu cầu đến `CanIf_Transmit()`.
4. `CanIf` tra cứu ID CAN và HTH từ bảng cấu hình, chuyển cấu trúc dữ liệu thành `Can_PduType` và gọi `Can_Write()` của CAN Driver.
5. Driver bxCAN ghi dữ liệu vào Hardware Mailbox. Khi hoàn thành gửi, driver gọi ngược lại chuỗi callback qua `CanIf_TxConfirmation → PduR_CanIfTxConfirmation → Com_TxConfirmation` để thông báo trạng thái truyền.

---

3. Kịch bản 2: Truyền chẩn đoán dữ liệu lớn phân mảnh qua CanTp

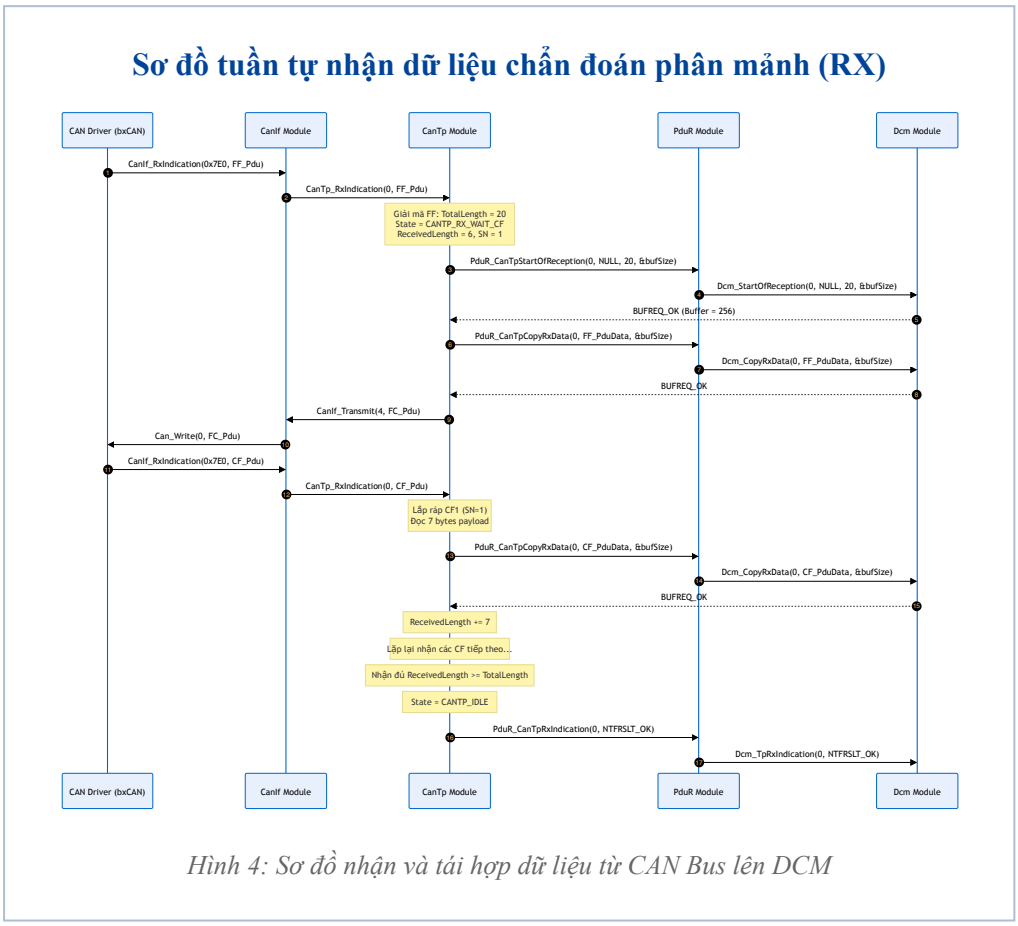
Kịch bản gửi gói dữ liệu chẩn đoán UDS có độ dài 20 bytes. Do kích thước lớn hơn 8 bytes, CanTp tự động phân đoạn thông điệp thành 1 First Frame (FF) và 2 Consecutive Frames (CF) với cơ chế điều khiển luồng (Flow Control CTS).



Hình 3: Sơ đồ phân mảnh truyền gói tin chẩn đoán lớn (>7 Bytes)

4. Kịch bản 3: Nhận chẩn đoán dữ liệu lớn phân mảnh qua CanTp

Kịch bản nhận và lắp ráp gói tin chẩn đoán 20 bytes từ bên ngoài bus gửi tới ECU để chuyển giao cho module DCM xử lý chẩn đoán.



Hình 4: Sơ đồ nhận và tái hợp dữ liệu từ CAN Bus lên DCM

5. Thuật toán trích xuất & xử lý bitwise byte điều khiển PCI

Byte đầu tiên của CAN frame (Byte 0) trong luồng chẩn đoán chứa thông tin định danh giao thức **PCI (Protocol Control Information)**.

5.1 Phân loại Frame

Để nhận diện loại frame nhận được, thuật toán áp dụng phép toán AND bitwise và dịch phải bit để lấy 4 bit cao nhất:

```
uint8 pciType = (CanTpRxPduPtr->SduDataPtr[0] & 0xF0) >> 4;
```

```
Ví dụ với byte đầu tiên là 0x1A (nhị phân: 0001 1010):
1. 0x1A & 0xF0 = 0x10 (0001 0000)
2. 0x10 >> 4 = 0x01 (0000 0001) → Đánh dấu loại First Frame (FF)
```

5.2 Trích xuất độ dài gói tin

- **Single Frame (SF)**: Độ dài được trích xuất trực tiếp từ 4 bit thấp của byte 0.

  ```
  uint8 sfLen = CanTpRxPduPtr->SduDataPtr[0] & 0x0F;
  ```

- **First Frame (FF)**: Do tổng độ dài gói tin có thể lên đến 4095 bytes, giá trị này sử dụng 12 bit (4 bit thấp của byte 0 và toàn bộ byte 1):

  ```
  uint8 fs = CanTpRxPduPtr->SduDataPtr[0] & 0x0F;
  uint8 lenLSB = CanTpRxPduPtr->SduDataPtr[1];
  uint16 totalRxLength = (fs << 8) | lenLSB;
  ```

---

6. Thuật toán phân mảnh dữ liệu phía phát (TX Segmentation)

Khi gửi dữ liệu lớn hơn 7 bytes, CanTp thực hiện thuật toán phân đoạn theo chu kỳ như sau:

6.1 Tăng tuần hoàn Sequence Number (SN)

Mỗi Consecutive Frame gửi đi bắt buộc phải được đánh số thứ tự từ 1 đến 15 rồi quay vòng về 0. Thuật toán sử dụng mặt nạ AND bitwise để tự động hóa việc quay vòng:

```
tx->SN = (tx->SN + 1) & 0x0F;
```

> Khi `tx->SN` đạt tới 15 (0x0F), việc cộng thêm 1 cho ra 16 (0x10). Phép toán `0x10 & 0x0F` sẽ trả về kết quả `0x00`, tự động đưa số thứ tự quay trở lại 0 theo chuẩn ISO-TP.

6.2 Cơ chế TX Pacing (Ngăn ngừa xung đột Bus)

Để ngăn chặn việc gửi dồn dập các frame gây nghẽn phần cứng Driver hoặc vi phạm luật thời gian tối thiểu `STmin` giữa các frame, CanTp sử dụng máy trạng thái bất đồng bộ:

- Khi frame cũ đang truyền trên bus, trạng thái kênh truyền được khóa ở `CANTP_TX_WAIT_CF_CONF`.
- Chỉ khi Driver phần cứng hoàn tất gửi và kích hoạt callback `CanTp_TxConfirmation()`, trạng thái mới được nâng lên `CANTP_TX_SEND_CF`.
- Hàm quét định kỳ `CanTp_MainFunction()` phát hiện trạng thái `CANTP_TX_SEND_CF` mới đóng gói dữ liệu và gửi frame tiếp theo, bảo đảm thời gian chạy an toàn cho Driver.

---

7. Thuật toán lắp ráp dữ liệu phía nhận (RX Reassembly)

Tại phía nhận, CanTp lắp ráp các frame dữ liệu nhận được để trả lại thông điệp nguyên vẹn.

7.1 Zero-Copy Buffer và định tuyến dữ liệu trực tiếp

Để giảm tiêu hao RAM cho vi điều khiển, CanTp RX không lưu trữ bản sao dữ liệu. Nó đóng vai trò là đường ống dẫn trực tiếp:

1. Khi nhận được First Frame, CanTp báo lên `PduR` bằng hàm `PduR_CanTpStartOfReception()` để yêu cầu tầng ứng dụng chuẩn bị không gian lưu trữ.
2. Với mỗi CF tiếp theo nhận được, dữ liệu được chuyển trực tiếp vào vùng đệm ứng dụng thông qua `PduR_CanTpCopyRxData()`.

7.2 Thuật toán bảo vệ chống tràn và cắt dữ liệu rác (Size Protection)

Ở gói Consecutive Frame cuối cùng, số byte dữ liệu nhận được có thể ít hơn 7 bytes. Thuật toán thực hiện tính toán số byte hữu ích thực tế và chặn tràn bộ đệm như sau:

```
// 1. Tính toán dung lượng payload thô (trừ 1 byte PCI)
uint8 len = CanTpRxPduPtr->SduLength - 1;

// 2. Chặn trên chống ghi đè dữ liệu rác ngoài tầm kích thước tổng t
if (RxState.ReceivedLength + len > RxState.TotalLength) {
    len = RxState.TotalLength - RxState.ReceivedLength;
}

// 3. Tiến hành copy lượng bytes đã tính toán lên buffer đích
rxDataInfo.SduLength = len;
PduR_CanTpCopyRxData(..., &rxDataInfo, ...);

// 4. Tích lũy dung lượng đã nhận thành công
RxState.ReceivedLength += len;
```

---

8. So sánh logic code hiện tại và tiêu chuẩn ISO 15765-2

Mã nguồn hiện tại trong project là phiên bản rút gọn phục vụ học tập và demo tính năng. Dưới đây là bảng đối chiếu thuật toán với tiêu chuẩn công nghiệp:

Tính năng	Tiêu chuẩn ISO-TP đầy đủ	Mã nguồn Demo hiện tại	Đánh giá rủi ro kỹ thuật
Kiểm tra thứ tự SN	Bắt buộc so sánh SN của CF nhận được có khớp với SN mong đợi hay không. Hủy và báo lỗi nếu lệch.	Bỏ qua việc đối chiếu SN. Có CF là copy thẳng dữ liệu vào bộ đệm chẩn đoán.	**Cao**: Nếu bị mất một CAN frame trên bus, dữ liệu ghép lại sẽ bị lệch và lỗi nội dung nhưng hệ thống không phát hiện được.
Kiểm soát BlockSize	Node phát phải dừng truyền CF khi đạt số lượng giới hạn gửi, chờ FC tiếp theo từ node nhận.	Lưu trữ tham số nhưng không đếm số lượng frame đã gửi để dừng. Truyền liên tục cho tới hết.	**Trung bình**: Sẽ gây tràn bộ đệm bên nhận nếu bên nhận có dung lượng RAM nhỏ hơn kích thước gói truyền.
Giãn cách STmin	Đo đạc thời gian giãn cách tối thiểu giữa 2 CF phát đi bảo đảm lớn hơn `STmin` bằng Hardware Timer.	Gửi CF tiếp theo ở chu kỳ quét quét nền tiếp theo của Main Function mà không so sánh thời gian.	**Thấp/Trung bình**: Có thể làm bên nhận bị quá tải ngắt nhận CAN do frame đến quá dồn dập.
ISO-TP Timers	Đếm ngược và quản lý nghiêm ngặt các bộ định thời `N_As`, `N_Bs`, `N_Cs`, `N_Ar`, `N_Br`, `N_Cr` để báo lỗi quá hạn.	Khai báo các trường timer trong cấu trúc trạng thái nhưng không có logic đếm ngược hay hủy kết nối.	**Trung bình**: Nếu một node đột ngột mất nguồn giữa phiên truyền, node còn lại sẽ bị kẹt vĩnh viễn trạng thái (Treo kênh).